

PrintLoop

Backend Implementation Priority List

Tasks ordered by completion level — April 2026

Executive Summary

The PrintLoop backend is approximately 46% complete across 6 development phases. This document lists all remaining backend tasks in order of how close they are to completion, starting with quick wins in Phase 4 (Kiosk API at 80% complete) and Phase 1 (Foundation at 85% complete), through to completely unbuilt systems in Phase 3 (Group Printing) and Phase 6 (Security & Testing). Total remaining effort: ~194 hours (~5 weeks for 1 developer, ~2.5 weeks for 2 developers).

Priority Icons

- P0 – Critical Blocks launch
- P1 – High Should complete this sprint
- P2 – Medium Next milestone
- P3 – Low Nice to have / future

Effort by Phase

Completion	Phase	Tasks	Effort
80%	Phase 4 — Kiosk API	5 tasks	~18h
85%	Phase 1 — Foundation	6 tasks	~9h
70%	Phase 2 — Customer App	6 tasks	~27h
20%	Phase 5 — Admin Panel	11 tasks	~48h
0%	Phase 3 — Group Printing	8 tasks	~36h
0%	Phase 6 — Security & Tests	12 tasks	~56h

Total backend remaining: ~194 hours (~5 weeks for 1 dev, ~2.5 weeks for 2 devs)

■ Phase 4 — Kiosk API (~80% complete)

Quick wins — all endpoints work, security gaps remain

Estimated effort: ~18h total (< 1 week)

■ Kiosk API key authentication — 4h

All /printer/* endpoints are PUBLIC. Anyone can mark jobs complete. Fix: Create kiosks table, add X-Kiosk-Key header validation middleware

src/middleware/kioskAuth.middleware.ts (new)

■ Server-side QR code generation — 3h

Customer gets 6-char PIN only. No QR image. Fix: Install qrcode npm, generate at job creation, store qrCodeUrl on PrintJob

src/modules/customer/services/printJob.service.ts

■ Kiosk heartbeat endpoint — 2h

No way to know which kiosks are online. Fix: Add GET /printer/heartbeat that updates kiosk.lastSeenAt

src/modules/printer/routes/printer.routes.ts

■ Group batch support in get-job — 5h

get-job returns single fileURL. Cannot handle GROUP_BATCH tokens. Fix: Return files[] array when jobType === GROUP_BATCH

src/modules/printer/services/printer.service.ts

■ Incremental page progress endpoint — 4h

Job status all-or-nothing. No partial-print resume. Fix: Add pagesCompleted column, PATCH /printer/progress endpoint

src/modules/printer/routes/printer.routes.ts

■ Phase 1 — Foundation (~85% complete)

Critical schema gaps — all entities exist, 6 columns missing

Estimated effort: ~9h total (< 2 days)

■ Create pricingConfig entity — 3h

Pricing hardcoded (■5 B/W, ■25 color). Cannot adjust per-page pricing without redeploy. Blocks admin panel pricing settings

src/database/entities/pricingConfig.entity.ts (new)

■ Create kiosks entity — 2h

No kiosks table. Cannot assign API keys or track online/offline status. Blocks kiosk API auth and admin kiosk management

src/database/entities/kiosk.entity.ts (new)

■ Add participantId to File entity — 1h

No way to trace group uploads to specific participant. Blocks Phase 3 group printing

src/database/entities/file.entity.ts

■ **Add perFilePrintConfig to File** — 1h

Batch printing applies uniform config. No per-file overrides. Blocks Phase 2 per-file batch UI

src/database/entities/file.entity.ts

■ **Extend AuditLog with resource columns** — 1h

Only logs userId + action. No context about what changed. Add: resourceType, resourceId, payload

src/database/entities/auditLog.entity.ts

■ **Add promo code fields to Promotion** — 1h

No customer coupon redemption possible. Add: promoCode (unique), maxUsage

src/database/entities/promotion.entity.ts

■ **Phase 2 — Customer App (~70% backend)**

High priority polish — core flow works, gaps in payments & notifications

Estimated effort: ~27h total (< 1 week)

■ **Move pricing to pricingConfig entity** — 3h

calculatePrice() uses hardcoded values. Fix: Query pricingConfig table, add GET /pricing/config endpoint. Depends on pricingConfig entity

src/modules/customer/services/printJob.service.ts

■ **Paystack webhook idempotency guard** — 3h

Duplicate events could mark payment complete multiple times. Fix: Store processed paymentReference, skip duplicates

src/modules/customer/controllers/webhook.controller.ts

■ **Email receipt after payment** — 4h

No email sent post-payment. Fix: Use nodemailer/Resend, trigger from webhook after marking PAID

src/modules/services/email.service.ts (new)

■ **GET /kiosks endpoint** — 3h

No kiosk finder. Frontend cannot show map/list. Fix: Add endpoint filtered by location. Depends on kiosks entity

src/modules/customer/routes/kiosk.routes.ts (new)

■ **Push notifications (FCM)** — 6h

Notification entity exists but no delivery. Fix: Integrate Firebase Admin SDK, trigger on PAID/PRINTING/COMPLETE

src/modules/services/notification.service.ts

■ **Stripe payment integration** — 8h

Only Paystack. International customers cannot pay. Fix: Implement PaymentService interface, toggle via env

src/modules/services/stripe.service.ts (new)

■ Phase 5 — Admin Panel (~20% backend)

Critical for operations — backend skeleton only, 0% frontend

Estimated effort: ~48h total (< 2 weeks)

■ AdminDashboardService aggregate metrics — 6h

Only returns totalUsers + recentUsers. Missing: revenue, pagesPrinted, onlineKiosks, jobsByStatus, revenueByDay. Use single SQL aggregate query

src/modules/admin/services/dashboard.service.ts

■ Pricing config CRUD endpoints — 4h

No admin routes to update pricing. Fix: GET/PATCH /admin/pricing. Depends on pricingConfig entity

src/modules/admin/routes/pricing.routes.ts (new)

■ Kiosk management endpoints — 5h

No way to list kiosks or set maintenance mode. Fix: GET /admin/kiosks, PATCH /admin/kiosks/:id/status

src/modules/admin/routes/kiosk.routes.ts (new)

■ Promotions CRUD endpoints — 5h

Promotion entity exists but no admin routes. Fix: Full CRUD + GET /admin/promotions/active for customer app

src/modules/admin/routes/promotion.routes.ts (new)

■ Refund endpoint — 5h

No refund logic. Fix: Call Paystack /refund API, store refundId, update status to REFUNDED

src/modules/admin/services/payment.service.ts

■ Requeue failed jobs — 3h

No way to retry failed jobs. Fix: PATCH /admin/print-jobs/:id/requeue, reset status to PENDING

src/modules/admin/services/printJob.service.ts

■ AdminRoles CRUD + RBAC middleware — 5h

AdminRole entity exists but no routes. No permission enforcement. Fix: Add role CRUD + middleware

src/modules/admin/routes/role.routes.ts (new)

■ Group session viewer endpoints — 4h

Fix: GET /admin/group-sessions with participant list

src/modules/admin/routes/groupSession.routes.ts (new)

■ AuditLog viewer endpoint — 3h

Fix: GET /admin/audit-logs, paginated, filterable. Depends on AuditLog extended columns

src/modules/admin/routes/auditLog.routes.ts (new)

■ Reports export endpoints — 6h

Fix: GET /admin/reports — CSV by date/campus/kiosk (pages, revenue, top users, promo impact)

src/modules/admin/routes/report.routes.ts (new)

■ User block/unblock endpoint — 2h

Fix: PATCH /admin/users/:id/status — blocked users cannot start jobs
src/modules/admin/services/user.service.ts

■ Phase 3 — Group Printing (0% built)

Full backend rewrite needed — schema ready, no service layer

Estimated effort: ~36h total (< 1 week if focused)

■ POST /customer/group-sessions — create — 6h

Host creates session with groupName, deadline, sharedSettings. Returns sessionId + share link (use nanoid for short IDs)
src/modules/customer/routes/groupSession.routes.ts (new)

■ POST /group-sessions/:id/join — 4h

Participant joins. Validates session OPEN and deadline not passed. Returns upload token
src/modules/customer/services/groupSession.service.ts (new)

■ POST /group-sessions/:id/upload — 6h

Participant uploads file. Generates watermarkId. Calculates cost. Queues watermarking BullMQ job (async)
src/modules/customer/services/groupSession.service.ts

■ GET /group-sessions/:id — host data — 4h

Returns participant list: name, fileUploaded, paymentStatus, cost
src/modules/customer/services/groupSession.service.ts

■ POST /group-sessions/:id/close — 5h

Host closes. Aggregates paid jobs. Generates batchToken. Sets status CLOSED. Reject if any unpaid (or allow override)
src/modules/customer/services/groupSession.service.ts

■ Scheduled cron — auto-close expired — 3h

BullMQ repeatable job every 15 min. Queries OPEN sessions where deadline < now. Auto-closes
src/modules/services/scheduler.service.ts (new)

■ PDF watermarking BullMQ worker — 8h

Fetches PDF from Cloudinary. Stamps watermarkId via pdf-lib. Re-uploads. Install pdf-lib. Subtle corner stamp
src/workers/watermark.worker.ts (new)

■ Phase 6 — Security & Testing (0% built)

Launch blockers — no tests, no file cleanup, missing security headers

Estimated effort: ~56h total (~2 weeks)

Security hardening (~20h)

- **Auto-delete files 24h post-completion** — 3h
BullMQ job on COMPLETE + 24h delay. Calls cloudinary.uploader.destroy(), nullifies fileURL in DB
src/workers/fileCleanup.worker.ts (new)
- **Helmet.js HTTP security headers** — 1h
npm install helmet → app.use(helmet()). One line fix with major impact
src/app.ts
- **Rate limiting on sensitive endpoints** — 3h
/auth/login: 5/min, /validate-code: 10/min, /printer/*: 100/min. Use express-rate-limit + Redis
src/middleware/rateLimit.middleware.ts (new)
- **Brute-force protection on codes** — 3h
5 wrong guesses from IP in 10 min → 429. Redis counter with TTL. Log to auditLog
src/modules/printer/services/printer.service.ts
- **Group session access control audit** — 3h
Participant from session A should not access session B. Enforce sessionId + participantToken validation
All group endpoints
- **SQL injection audit** — 4h
Full audit of TypeORM usage. Use parameterised queries exclusively. Never concatenate user input
All service files
- **NDPR / GDPR policy document** — 2h
Files: 24h. PII: anonymise after 12mo. Audit logs: 2yr. Required for investor due diligence
PRIVACY_POLICY.md

Testing infrastructure (~36h)

- **Jest unit test setup** — 6h
Add Jest config, test utilities, DB mocking
jest.config.js (new)
- **Unit tests — PrintJobService** — 8h
Test: createPrintJob, calculatePrice, code uniqueness. Target >80% coverage
src/modules/customer/services/printJob.service.spec.ts (new)
- **Unit tests — PaymentService + webhook** — 6h
Test: createPaymentRequest, handleWebhook (success/duplicate/tampered), refund
src/modules/services/payment.service.spec.ts (new)
- **Unit tests — printer.service** — 5h
Test: validateCode, complete, fail, progress
src/modules/printer/services/printer.service.spec.ts (new)

■ **E2E test — full print flow — 12h**

Playwright: register → upload → pay (Paystack test) → code → kiosk validate → complete
e2e/printFlow.spec.ts (new)

■ Recommended Backend Priority Order

Execute in this sequence for fastest path to launch-ready backend:

Week	Focus	Tasks	Effort
Week 1	Security + Schema	Tasks #1-11: kiosk auth, QR codes, pricing/kiosk entities, schema gaps	~25h
Week 2	Customer Polish	Tasks #12-17: pricing DB integration, webhook idempotency, email receipts	~27h
Week 3	Admin Backend	Tasks #18-28: dashboard stats, pricing CRUD, kiosk mgmt, refunds, RBAC	~48h
Week 4	Group Printing	Tasks #29-36: full group session API + watermarking	~36h
Week 5	Testing	Tasks #37-48: file cleanup, Helmet, rate limits, Jest setup, unit tests	~48h

This sequence delivers the highest-value improvements first: fixes critical security gaps (week 1), polishes the customer-facing flow (week 2), enables admin operations (week 3), unlocks the group printing revenue stream (week 4), and establishes testing infrastructure to prevent regressions (week 5). By week 3, you have a production-ready single-print platform with full admin control. By week 5, you have the complete feature set with test coverage.